```
# ================================
# Online Payments Fraud Detection
# ================================

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_s

# ------------------------------
# Step 1: Load the Dataset
# ------------------------------
# Replace with your actual dataset filename (CSV from Kaggle)
df = pd.read_csv("online_payments_fraud.csv")

# Display first few rows
print("Dataset Preview:")
print(df.head())

# ------------------------------
# Step 2: Data Preprocessing
# ------------------------------

# Drop unnecessary columns (nameOrig, nameDest — as they are identifiers)
df = df.drop(['nameOrig', 'nameDest'], axis=1)

# Encode 'type' (categorical feature)
le = LabelEncoder()
df['type'] = le.fit_transform(df['type'])

# Check for missing values
print("\nMissing Values:\n", df.isnull().sum())

# Drop rows with missing values
df.dropna(inplace=True)

# ------------------------------
# Step 3: Define Features and Target
# ------------------------------
X = df.drop('isFraud', axis=1)
y = df['isFraud']

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stra

print("\nTraining data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)

# ------------------------------
# Step 4: Train the Model
# ------------------------------
model = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
model.fit(X_train, y_train)

# ------------------------------
# Step 5: Evaluate the Model
# ------------------------------
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Accuracy and ROC-AUC
```

```
acc = accuracy_score(y_test, y_pred)
roc = roc_auc_score(y_test, y_prob)

print("\nModel Performance:")
print(f"Accuracy: {acc*100:.2f}%")
print(f"ROC-AUC: {roc:.4f}")

# Confusion Matrix and Classification Report
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ------------------------------
# Step 6: Feature Importance
# ------------------------------
importances = pd.Series(model.feature_importances_, index=X.columns)
importances.sort_values(ascending=False).plot(kind='bar', figsize=(8, 4), title='Feature Impor
plt.tight_layout()
plt.show()

# ------------------------------
# Step 7: Save Model (optional)
# ------------------------------
import joblib
joblib.dump(model, 'fraud_detection_model.pkl')

print("\nModel saved as 'fraud_detection_model.pkl'")
```

```
Dataset Preview:
   step      type    amount     nameOrig  oldbalanceOrg  newbalanceOrig  \
0     1   PAYMENT   9839.64  C1231006815       170136.0       160296.36
1     1   PAYMENT   1864.28  C1666544295        21249.0        19384.72
2     1  TRANSFER    181.00  C1305486145          181.0            0.00
3     1  CASH_OUT    181.00   C840083671          181.0            0.00
4     1   PAYMENT  11668.14  C2048537720        41554.0        29885.86

      nameDest  oldbalanceDest  newbalanceDest  isFraud  isFlaggedFraud
0  M1979787155             0.0             0.0      0.0             0.0
1  M2044282225             0.0             0.0      0.0             0.0
2   C553264065             0.0             0.0      1.0             0.0
3    C38997010         21182.0             0.0      1.0             0.0
4  M1230701703             0.0             0.0      0.0             0.0

Missing Values:
 step              0
type              0
amount            1
oldbalanceOrg     1
newbalanceOrig    1
oldbalanceDest    1
newbalanceDest    1
isFraud           1
isFlaggedFraud    1
dtype: int64

Training data shape: (316654, 8)
Testing data shape: (79164, 8)

Model Performance:
Accuracy: 99.96%
ROC-AUC: 0.9381

Confusion Matrix:
 [[79122     1]
 [   27    14]]

Classification Report:
              precision    recall  f1-score   support

         0.0       1.00      1.00      1.00     79123
         1.0       0.93      0.34      0.50        41
```